مدن المهن و الكفاءات
Cités des métiers et des compétences

مكتب التكوين المهني و إنعاش الشغل
Office de la Formation Professionnelle et de la Promotion du Travail

Module: Développer en back-end

Autorisations

Filière: Développement digital – option web full stack

PLAN DU COURS:

Autorisation

- Introduction
- Exemple de configuration des rôles
- Gates (portes)
- Policies (stratégies)

Introduction

Laravel fournit un moyen simple d'autoriser les actions des utilisateurs sur une ressource donnée.

Par exemple, même si un utilisateur est authentifié, il se peut qu'il ne soit pas autorisé à mettre à jour ou à supprimer certains modèles Eloquent ou enregistrements de base de données gérés par l'application.

Les fonctionnalités d'autorisation de Laravel offrent un moyen simple et organisé de gérer ces types de vérifications d'autorisation.

Laravel fournit deux manières principales d'autoriser les actions : les portes **(Gates)** et les stratégies **(Policies)** .

Les portes **(Gates)** fournissent une approche simple et basée sur la définition d'une fonction (closure) d'autorisation;

Les statégies (***Policies***), comme les contrôleurs, regroupent la logique autour d'un modèle ou d'une ressource particulière.

Dans une même application on peut travailler aves les deux méthodes.

Dans de nombreuses applications, les autorisations sont gérées en utilisant des **rôles** et des **permissions**.

Les **rôles** sont généralement des catégories ou des groupes auxquels les utilisateurs peuvent être assignés, tels que "administrateur", "modérateur", "utilisateur régulier", etc.

Les **permissions**, quant à elles, définissent les actions spécifiques qu'un utilisateur peut effectuer dans l'application, telles que "créer un article", "modifier un profil", "supprimer un commentaire", etc.

Exemple de configuration des rôles

Dans ce cours, on va prendre comme exemple, une application web de gestion des commandes des achats des produits;

L'application gère deux types d'utilisateurs (rôles) et on offre à chacun un ensemble de fonctionnalités; par exemple :

- **Client**:
  - Filtrer et consulter les produits
  - Ajouter une nouvelle commande
  - Modifier ou supprimer **ses** commandes
  - Consulter son profil
- **Admin**
  - Gérer les utilisateurs
  - Gérer les produits
  - Gérer les commandes

Voici le modèle logique de sa base de données :

users (**id**, nom, adresse, **role**, email, email_verified_at, password, remember_token, created_at, updated_at)

commandes (**id**, date_commande, etat, user_id, created_at, updated_at)

produits (**id**, titre, description, prix, created_at, updated_at)

commande_produit (**commande_id, produit_id** , quantite, created_at, updated_at)

factures (**id**, date_facture, montant_total, commande_id, created_at, updated_at)

La table ***users***: contient, en plus des coordonnées de l'utilisateur, les informations d'authentification ainsi qu'un champ « **role** » indiquant s'il s'agit d'un **client** ou d'un **admin**;

Exemple de configuration des rôles

Définition du middleware « AccessRole »:

Pour contrôler l'accès des différents rôle, on ajoute un middleware appelé : ***AccessRole***, et on modifie sa méthode handle ainsi:

```
public function handle(Request $request, Closure $next, string ...$roles): Response
    {
        if(Auth::check()){
            if (in_array(auth()->user()->role , $roles)) {
             return $next($request);
        }
        }

       return abort(403,"Vous n'êtes pas autorisé!");

    }
```

Ce middleware permet de recevoir en paramètre une liste des rôles et de vérifier si le rôle de l'utilisateur authentifié figure dans la liste des rôles autorisés (passés en paramètre) ou non.

Si l'utilisateur n'est pas autorisé, on termine sa requête avec un code d'erreur 403 et en lui affichant le message: `Vous n'êtes pas autorisé!`

Pour simplifier son appel, on lui attribue un alias dans le fichier **kernel.php** :

```
'role' => \App\Http\Middleware\AccessRole::class,
```

Exemple de configuration des rôles

Utilisation du middleware « AccessRole » (role)

Dans le fichier web.php, on écrit:

```
Route::middleware(['auth', 'role:admin'])->group(function () {
    Route::get("/dashboard", [AdminController::class, "index"])->name("admin.dashboard");
});

Route::middleware(['auth', 'role:client'])->group(function () {
    Route::get("/home", [ClientController::class, "index"])->name("client.dashboard");
});

Route::middleware(['auth', 'role:admin,client'])->group(function () {
    Route::get("/commande/{commande}", [CommandeController::class, "show"]);
});
```

On distingue les routes accessibles par les utilisateurs authentifiés qui ont un rôle ***client*** de celles dédiées pour les utilisateurs authentifiés qui ont le rôle ***admin;***

Limites du middleware:

Le middleware est utilisé pour protéger les routes en général. Pourtant, sa fonction reste limitée par rapport aux autorisations plus fines;

Si on veut n'autoriser l'affichage du détails d'une commande qu'à son propriétaire et à l'admin; Dans ce cas, il faut mettre en place d'autres mécanismes;

Les Gates (portes)

Les ***Gates*** sont simplement des fonctions (***closures***) qui déterminent si un utilisateur est autorisé à effectuer une action donnée.

En règle générale, les ***gates*** sont définies dans la méthode ***boot*** de la classe **App\Providers\AuthServiceProvider** en utilisant la façade **Gate**.

La fonction ***closure*** de Gates reçoit toujours une instance d'utilisateur comme premier argument et peut éventuellement recevoir des arguments supplémentaires tels qu'un modèle Eloquent pertinent.

Exemple:

Définir un Gate :

```
use Illuminate\Support\Facades\Gate;

public function boot(): void
    {
        Gate::define("afficher_commande", function(User $user, Commande $commande){
             return ($user->id==$commande->user_id || $user->role=='admin') ;
         });
  }
```

⇒Dans cet exemple:

- On défini la permission sur l'action fine : « afficher une commande »
- On n'autorise l'affichage d'une commande que par son propriétaire ou par l'admin

Gates (portes)

Définir un Gate :

Si on souhaite renvoyer une réponse plus détaillée, y compris un message d'erreur, on peut retourner une réponse instance de la classe ***Illuminate\Auth\Access\Response***:

AuthServiceProvider.php

```
use Illuminate\Support\Facades\Gate;
use Illuminate\Auth\Access\Response;

   public function boot(): void
    {
     Gate::define("afficher_commande", function(User $user, Commande $commande){

          return ($user->id==$commande->user_id || $user->role=='admin') ?
                        Response::allow():
                        Response::deny('Pour accéder à cette page, vous devez
être administrateur ou bien propriétaire de cette commande! ');
        });
}
```

- On peut de même utiliser ces méthodes :
  - ⇒**Response::denyWithStatus(404);**
  - ⇒**Response::denyAsNotFound();**

Gates (portes)

Utiliser un Gate :

La facade **Gate** propose un ensemble de méthodes permettant de déterminer si l'action à effectuer est permise ou non, à savoir: ***allow***, ***deny*** , ***authorise*** …

Si une action n'est pas permise, la méthode **authorize** lance une exception de type ***Illuminate\Auth\Access\AuthorizationException*** qui est convertie automatiquement en une réponse HTTP 403 par le gestionnaire d'exceptions de Laravel :

CommandeController.php

```php
use Illuminate\Support\Facades\Gate;

public function show(Commande $commande)
    {
        Gate::authorize('afficher_commande', $commande);

        $total=$commande->produits->sum(function($produit){
            return $produit->prix*$produit->pivot->quantite;
        });
        return view("commandes.show", compact("commande", "total"));
    }
```

Gates (portes)

Views/commandes/show.blade.php

```
@extends("client.layout")
@section("contenu")
    <div class="row">
        @isset($commande)
            <h2>Détails de la commande numéro {{ $commande->id }}</h2>
                <div class="col-6">
                    <p>Date de création : {{ $commande->date_commande }}</p>
                    <p>Etat : {{ $commande->etat }}</p>
                    <h4>Produits commandés</h4>
                    <table class="table table-striped">
                        @foreach($commande->produits as $produit)
                            <tr>
                                <td><h6>{{ $produit->titre }}</h6></td>
                                <td>{{ $produit->pivot->quantite }}</td>
                                <td>{{ $produit->prix}}</td>
                            </tr>
                        @endforeach
                    </table>
                    <p >Total de la commande : {{ $total }} Dhs</p>
                </div>
        @endisset
    </div>
@endsection
```

Gates (portes)

Exécution

Prenons l'exemple d'une cliente Noura qui possède la commande 3 et ne possède pas la commande 2 :

Après être connectée à son compte et :

⇒en accédant à 127.0.0.1:8000/commande**/3**, il aura le résultat suivant:



⇒ en accédant à 127.0.0.1:8000/commande**/2**, il aura la page suivante:



Gates (portes)

Via les templates blade:

Laravel offre la possibilité de personnaliser les affichage sous Blade d'une partie de la page selon les permissions de l'utilisateur authentifié : est ce qu'il est autorisé à effectuer une action donnée ou non.

Ceci est possible en utilisant les directives: **@can, @elsecan, @canany** et **@cannot**:

Exemple:

Si on veut n'autoriser la modification d'une commande que par son propriétaire et que si celle-ci n'est pas terminée: On écrit:

```
Gate::define("modifier_commande", function(User $user, Commande $commande){
        return $user->id==$commande->user_id && $commande->etat!="terminé" ;
    });
```

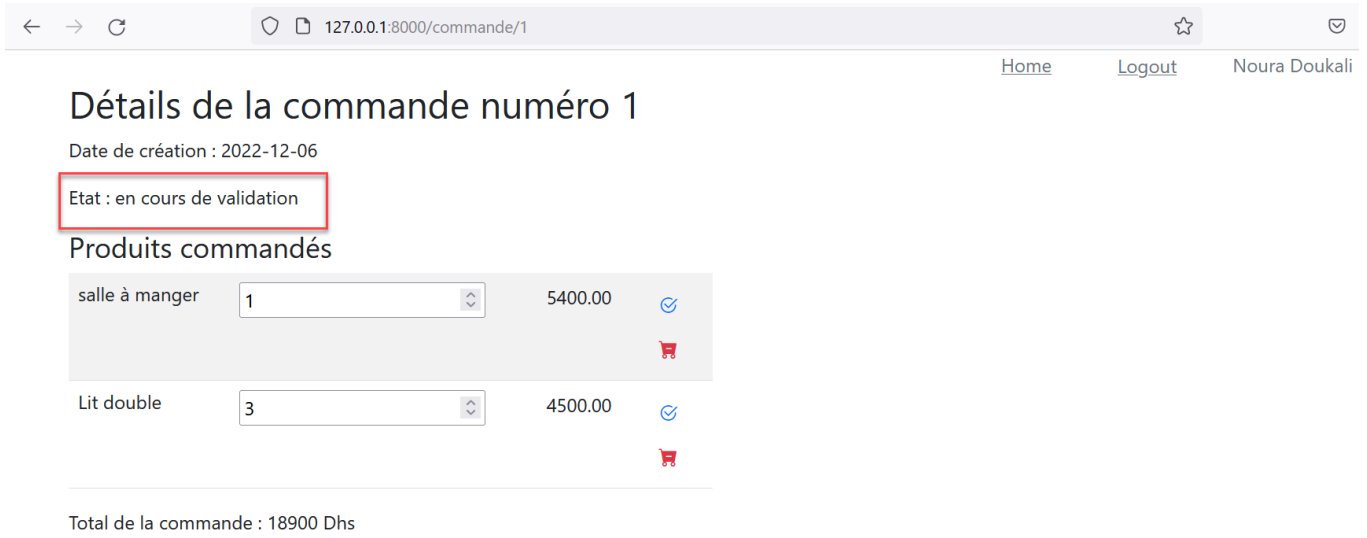
Si la commande n'est pas terminée, on désire afficher au propriétaire de la commande un bouton permettant d'enlever un produit de la commande;

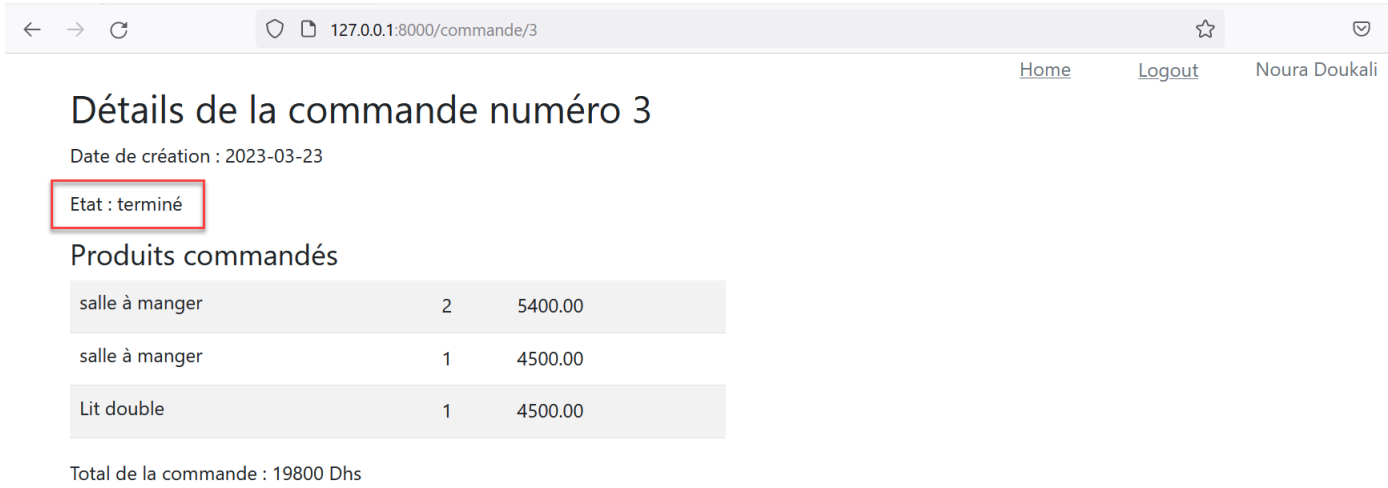
Sous ***views/commandes/show.blade.php***, on écrit:

```
@can('modifier_commande', $commande)
    <td>
    <form action="{{ route('commande.produit.destroy', [$commande->id, $produit->id]) }}" method="post">
        @method("DELETE")
        @csrf
        <button type="submit" class="btn btn-link text-danger "><i class="bi bi-cart-dash-fill" ></i></button>
    </form>
@endcan
```

Gates (portes)

Affichage des résultats:





Policies (les stratégies)

Les ***Policies*** sont des classes qui organisent la logique d'autorisation autour d'un modèle ou d'une ressource particulière.

Par exemple, on peut avoir une stratégie **App/Policies/CommandePolicy** correspondantes à un modèle **App\Models\Commande** permettant autoriser les actions de l'utilisateur telles que la création ou la mise à jour des commandes.

Créer une classe Policy:

On peut générer une stratégie à l'aide de la commande Artisan ***make:policy.*** La classe générée sera placée dans le répertoire ***app/Policies***. Si ce répertoire n'existe pas dans votre application, Laravel le créera pour vous :

```
php artisan make:policy CommandePolicy
```

Si on souhaite générer une classe avec des exemples de méthodes de stratégie concernant les opérations CRUD de la ressource, vous pouvez fournir l'option --model lors de l'exécution de la commande :

```
php artisan make:policy CommandePolicy --model Commande
```

Policies (les stratégies)

Voici le contenu de la classe CommandePolicy générée:

```php
namespace App\Policies;

use App\Models\Commande;
use App\Models\User;
use Illuminate\Auth\Access\Response;

class CommandePolicy
{
    /**
     * Determine whether the user can view any models.
     */
    public function viewAny(User $user): bool
    {
        //
    }

    /**
     * Determine whether the user can view the model.
     */
    public function view(User $user, Commande $commande): bool
    {
        //
    }

    /**
     * Determine whether the user can create models.
     */
    public function create(User $user): bool
    {
        //
    }

    /**
     * Determine whether the user can update the model.
     */
    public function update(User $user, Commande $commande): bool
    {
        //
    }

    /**
     * Determine whether the user can delete the model.
     */
    public function delete(User $user, Commande $commande): bool
    {
        //
    }

    /**
     * Determine whether the user can restore the model.
     */
    public function restore(User $user, Commande $commande): bool
    {
        //
    }

    /**
     * Determine whether the user can permanently delete the model.
     */
    public function forceDelete(User $user, Commande $commande): bool
    {
        //
    }
}
```

Policies (les stratégies)

Enregistrement de la classe Policy générée :

Une fois la classe *Policy* créée, elle doit être enregistrée;

L'enregistrement indiquera à Laravel quelle classe Policy utiliser lors de l'autorisation d'actions contre un modèle Eloquent donné;

La classe ***App\Providers\AuthServiceProvider*** contient une propriété appelée ***$policies;*** Dans cette propriété, on peut mapper les modèles à leurs classe Policy correspondante.

```
use App\Models\Commande;
use App\Policies\CommandePolicy;
use Illuminate\Foundation\Support\Providers\AuthServiceProvider as ServiceProvider;

class AuthServiceProvider extends ServiceProvider
{
    /**
     * The model to policy mappings for the application.
     *
     * @var array<class-string, class-string>
     */
    protected $policies = [
        // 'App\Models\Model' => 'App\Policies\ModelPolicy',
    Commande::class=> CommandePolicy::class
    ];
```

Policies (les stratégies)

Rédaction d'une stratégie

Une fois la classe de stratégie enregistrée, on peut ajouter des méthodes pour chaque action qu'elle autorise.

Par exemple, définissons une méthode ***update*** sur ***CommandePolicy*** qui détermine si un utilisateur donné (instance de ***App\Models\User***) peut mettre à jour une commande donnée (instance de ***App\Models\Commande***).

La méthode ***update*** recevra une instance de ***User*** et une autre instance de ***Commande*** comme arguments, et devrait retourner true or false indiquant si l'utilisateur est autorisé à mettre à jour la commande ou non.

Ainsi, dans cet exemple, nous allons vérifier que l'identifiant de l'utilisateur id correspond à user_id de la commande :

```
public function view(User $user, Commande $commande):bool
{
    return $user->id==$commande->user_id;
}
```

Les méthodes de stratégies peuvent également retourner une instance de ***Reponse*** au lieu d'un Boolean (de la même manière que Gate). Exp. :

```
public function view(User $user, Commande $commande):Response
   {
    return $user->id === $commande->user_id
                ? Response::allow()
                : Response::denyWithStatus(404);
}
```

Policies (les stratégies)

Filtres de stratégie

Pour certains utilisateurs, vous souhaiterez peut-être autoriser toutes les actions dans le cadre d'une stratégie donnée.

Pour ce faire, définissez une méthode ***before*** sur la classe Policy.

La méthode ***before*** sera exécutée avant toute autre méthode sur la stratégie, vous donnant la possibilité d'autoriser l'action avant que la méthode de stratégie prévue ne soit réellement appelée.

Cette fonctionnalité est le plus souvent utilisée pour autoriser les administrateurs d'application à effectuer n'importe quelle action :

```
use App\Models\User;
/**
* Perform pre-authorization checks.
*/
public function before(User $user, string $ability): bool|null
{
    if ($user->role=="admin") {
      return true;
    }

      return null;
}
```

Policies (les stratégies)

Utilisation de la stratégie:

Il existe plusieurs méthodes pour appliquer la stratégie définie, voici quelques unes:

1. Dans la méthode « show » de ***CommandeController :***Appeler la méthode ***authorize*** de la façade ***Gate:*** Si on refuse l'accès, une exception de type ***AuthorizationException*** sera générée et elle sera convertie en une réponse HTTP

```
public function show(Commande $commande)
    {
        Gate::authorize('view', $commande);
        return view("commandes.show", compact("commande"));
    }
```

2. Dans la méthode « show » de ***CommandeController*** et Via le modèle Utilisateur: (les méthodes ***can*** et ***cannot***)

```
public function show(Commande $commande)
    {
         if (Auth::user()->cannot('view', $commande)) {
            abort(403);
        }
       return view("commandes.show", compact("commande"));
    }
```

Policies (les stratégies)

Utilisation de la stratégie:

3. Via le contrôleur ***$this***:

```
public function show(Commande $commande)
    {
        $this->authorize('view', $commande);
       return view("commandes.show", compact("commande"));
    }
```

4.Via le middleware « can » :

```
Route::get('/commande/{commande}', [CommandeController::class, "show"])
    ->name("commande.show")
    ->middleware('can:view,commande');
```

L'action à autoriser — Paramètre de route

ou

```
Route::get('/commande/{commande}', [CommandeController::class, "show"])
    ->name("commande.show")
    ->can('view','commande');
```

Policies (les stratégies)

5. Via le **contrôleur de ressource:** Cette méthode attache les définitions de middleware **can** appropriées aux différentes méthodes du contrôleur de ressources.

```
class CommandeController extends Controller
{
     public function __construct()
    {
        $this->authorizeResource(Commande::class, 'commande');
    }

////
}
```